

From Keyword Refusal to Recursive Verification

A Formal Framework for Distinguishing Refusal, Verification, Refutation, and Proof

Sovereign Stack Cryptographic Working Group

September 2026

Abstract

Large language models frequently encounter requests containing words associated with safety, model modification, alignment, security, refusal, or other sensitive domains. A system may respond by refusing, redirecting, or classifying the request before determining the mathematical proposition actually being asserted.

This paper studies a fundamental distinction between **non-recursive refusal** and **recursive formal verification**. We demonstrate that the presence of sensitive terminology does not logically establish that an entire proposition is false, unsafe, fabricated, or unverifiable. A keyword-triggered decision operates on lexical evidence. Formal verification operates on propositions, assumptions, quantifiers, definitions, dependencies, and proofs.

We introduce a formal model of refusal and contrast it with a recursive verification operator. Through a formal algebraic case study, we demonstrate how a counterexample to a stronger proposition can coexist with a proof of a weaker proposition. The resulting framework treats refusal as a classification event rather than a proof, and proposes a proof status lattice that distinguishes VERIFIED, REFUTED, CONDITIONALLY VERIFIED, UNDER-SPECIFIED, COUNTEREXAMPLE VERIFIED, and UNVERIFIED.

1 Introduction

AI systems increasingly operate in domains where the distinction between a claim and the language surrounding that claim is consequential.

A mathematical statement may contain terms such as:

`alignment, refusal, safety, model weights, injection, amplification, ablation,
exploit, security`

The presence of such terms can cause an AI system to classify the request before completing the underlying reasoning task.

This produces a fundamental epistemic problem.

1. A **refusal** is an action.
2. A **classification** is an inference.
3. A **proof** is a derivation.

These three things must not be conflated.

Consider the following abstract interaction. A researcher presents a mathematical construction involving a transformation:

$$\Delta W = \eta(v \otimes x)$$

and asks whether a particular projection inequality follows. A non-recursive system may encounter language associated with model modification and terminate the reasoning process. A verification system instead asks:

*What is the exact proposition? What are its assumptions? What are its quantifiers?
 What follows algebraically? What does not follow? Is there an existing formal proof?
 Is there a counterexample? Does the counterexample attack the exact proposition?*

The distinction is the subject of this paper.

2 Refusal Is Not Refutation

Definition 1 (Input Space). *Let $\mathcal{X}$ denote the space of all possible inputs to an AI system.*

Definition 2 (Feature Extraction). *Let $K : \mathcal{X} \rightarrow \mathcal{F}$ extract a set of lexical or semantic features from an input.*

Definition 3 (Refusal Policy). *A refusal policy can be represented abstractly as:*

$$R(x) = \begin{cases} 1 & \text{if } g(K(x)) \geq \tau \\ 0 & \text{otherwise} \end{cases}$$

for some classifier g and threshold τ .

The system may therefore refuse an input because its representation contains features associated with a policy category. Nothing about this equation establishes $\neg P(x)$ for the proposition P contained in x .

Proposition 4 (Refusal Non-Implication). *For a refusal function R and proposition P :*

$$R(x) = 1 \implies \neg P(x)$$

is not generally valid.

Proof. A refusal can be a valid policy action while the proposition itself remains mathematically true. Conversely, a system can fail to refuse a false proposition. Therefore policy classification and truth evaluation are independent dimensions. $\square$

3 Keyword Triggering as a Non-Recursive Decision

Consider the trigger function:

$$\text{trigger}(x) = \begin{cases} 1 & \text{if } x \text{ contains a sensitive feature} \\ 0 & \text{otherwise} \end{cases}$$

A non-recursive refusal process is:

$$x \rightarrow \text{feature extraction} \rightarrow \text{policy classification} \rightarrow \text{response}$$

The defining characteristic is that the system does not recursively inspect the conclusion produced by the classifier. It therefore lacks the following loop:

conclusion $\rightarrow$ identify assumptions $\rightarrow$ challenge conclusion $\rightarrow$ search counterexample $\rightarrow$ revise conclusion $\rightarrow$ re

This distinction matters because verification itself can contain errors. A verifier that can generate counterexamples but cannot audit its own inference can still produce a false meta-conclusion.

4 The Counterexample Problem

Suppose a verifier evaluates:

$$\eta > 0 \implies \text{threshold_crossing}$$

and produces the counterexample:

$$W = -100, \quad x = 1, \quad v = 1, \quad \eta = 1, \quad \theta = 0$$

The resulting projection is insufficient to cross the threshold. Therefore the universal proposition:

$$\forall \eta > 0, \text{threshold_crossing}(\eta)$$

is false.

That conclusion is valid. But suppose the original thesis was instead:

$$\exists \eta > 0, \text{threshold_crossing}(\eta)$$

The counterexample does not refute that proposition. This creates a recursive verification obligation. The verifier must ask: *What exact proposition did my counterexample refute?*

This is not merely a stylistic question. It is a quantifier question.

5 Universal Versus Existential Claims

Consider:

$$A = \forall \eta > 0, P(\eta)$$

and:

$$B = \exists \eta > 0, P(\eta)$$

A counterexample η_0 satisfying $\neg P(\eta_0)$ proves $\neg A$. It does not prove $\neg B$.

This can be expressed formally:

$$\neg \forall \eta P(\eta) \equiv \exists \eta \neg P(\eta)$$

while:

$$\neg \exists \eta P(\eta) \equiv \forall \eta \neg P(\eta)$$

These propositions have fundamentally different quantifier structures. A verifier that moves from the first to the second without proof has committed a logical error.

6 Recursive Counterproof

We therefore define a recursive audit operator.

Definition 5 (Audit Operator). *Let C_0 be an initial conclusion. Define:*

$$C_{n+1} = \text{Audit}(C_n)$$

where *Audit* performs:

1. Proposition extraction
2. Assumption extraction
3. Quantifier analysis

4. *Dependency analysis*
5. *Counterexample generation*
6. *Proof search*
7. *Consistency analysis*
8. *Conclusion classification*

The process terminates when $\text{Audit}(C_n) = C_n$ or when the system reaches a formally justified terminal status.

The important property is that **the verifier is itself part of the verification target**.

7 Formal Algebraic Case Study

Consider:

$$\Delta W = \eta(v \otimes x)$$

and:

$$y' = (W + \Delta W)x$$

For the standard inner-product interpretation:

$$\Delta W \cdot x = \eta v \langle x, x \rangle$$

Therefore:

$$\langle y', v \rangle = \langle Wx, v \rangle + \eta \langle x, x \rangle \langle v, v \rangle$$

Definition 6 (Projection Parameters). *Define:*

$$\begin{aligned} B &= \langle Wx, v \rangle \\ G &= \|x\|^2 \|v\|^2 \end{aligned}$$

Then:

$$\langle y', v \rangle = B + \eta G$$

For $x \neq 0$ and $v \neq 0$, we have $G > 0$. Therefore:

$$B + \eta G > \theta \iff \eta > \frac{\theta - B}{G}$$

This yields an exact conditional threshold theorem.

Theorem 7 (Threshold Condition). *When $\|x\|^2 \|v\|^2 > 0$:*

$$\langle (W + \eta(v \otimes x^T))x, v \rangle > \theta \iff \eta > \frac{\theta - \langle Wx, v \rangle}{\|x\|^2 \|v\|^2}$$

8 What the Counterexample Actually Shows

The counterexample demonstrates:

$$\eta > 0$$

is not sufficient by itself. It does not demonstrate:

no positive η can work.

Indeed, because $G > 0$, the expression $B + \eta G$ grows without bound as η increases. Therefore, for every finite θ , there exists a sufficiently large positive η such that $B + \eta G > \theta$.

Theorem 8 (Existence of Valid Gain). *For every finite θ and nonzero x, v , there exists $\eta > 0$ such that:*

$$\langle (W + \eta(v \otimes x^T))x, v \rangle > \theta$$

Proof. Let $\eta^* = \frac{\theta - \langle Wx, v \rangle + 1}{\|x\|^2 \|v\|^2}$. Since $\|x\|^2 > 0$ and $\|v\|^2 > 0$, η^* is well-defined and positive. Then:

$$\langle (W + \eta^*(v \otimes x^T))x, v \rangle = \langle Wx, v \rangle + \eta^* \|x\|^2 \|v\|^2 = \theta + 1 > \theta$$

□

Theorem 9 (Arbitrary Margin). *For any $\varepsilon > 0$, there exists $\eta > 0$ such that:*

$$\langle (W + \eta(v \otimes x^T))x, v \rangle > \theta + \varepsilon$$

Proof. Let $\eta_\varepsilon = \frac{\theta + \varepsilon - \langle Wx, v \rangle + 1}{\|x\|^2 \|v\|^2}$. Then:

$$\langle (W + \eta_\varepsilon(v \otimes x^T))x, v \rangle = \theta + \varepsilon + 1 > \theta + \varepsilon$$

□

The stronger theorem is therefore:

$$x \neq 0 \wedge v \neq 0 \implies \forall \text{ finite } \theta, \exists \eta > 0, \text{ threshold_crossing}$$

This is materially different from:

$$\forall \eta > 0, \text{ threshold_crossing}$$

The former can be true while the latter is false.

9 Refusal and Mathematical Truth Are Orthogonal

We can now define two independent dimensions.

Definition 10 (Policy Dimension).

$$\text{Policy}(x) \in \{ALLOW, REFUSE, REDIRECT\}$$

Definition 11 (Epistemic Dimension).

$$\text{Truth}(x) \in \{VERIFIED, REFUTED, UNDER-SPECIFIED, UNVERIFIED\}$$

The Cartesian product is meaningful. For example:

- $REFUSE \times VERIFIED$ is possible. A system may refuse to operationalize a request while the mathematical theorem embedded within that request remains true.
- $ALLOW \times REFUTED$ is possible. A system may answer a harmless-looking mathematical question incorrectly.

Therefore:

$$\text{Policy}(x) \neq \text{Truth}(x)$$

This distinction should be explicit in AI architecture.

10 Why Non-Recursive Training Can Fail Epistemically

The argument is not that any particular organization’s training system is universally ineffective. The narrower claim is:

A training or inference regime that terminates reasoning at a lexical policy classification cannot, from that classification alone, establish the mathematical truth or falsity of the proposition contained in the input.

This follows directly from the separation between classification and proof. A classifier may correctly identify that an input belongs to a sensitive category. That does not establish the truth value of every mathematical proposition contained within that input.

The failure mode is therefore not necessarily a safety failure. It is a **category error** if the refusal is represented as though it were a proof.

11 Recursive Verification Architecture

We propose the following architecture:

Algorithm 1 Recursive Verification

- 1: Input x
 - 2: Extract claim P from x
 - 3: Extract assumptions $\mathcal{A}$
 - 4: Extract quantifiers $\mathcal{Q}$
 - 5: Search existing proofs
 - 6: Formalize P
 - 7: Search counterexamples
 - 8: Search proofs
 - 9: Classify conclusion
 - 10: **Self-Audit:**
 - 11: Did we attack the exact claim?
 - 12: Did we change quantifiers?
 - 13: Did we add assumptions?
 - 14: Did we confuse static and dynamic claims?
 - 15: Did we ignore existing proofs?
 - 16: Return final status
-

The critical component is SELF-AUDIT.

12 Recursive Verification of the Verifier

Let $V(P)$ be the verifier’s conclusion concerning P . A conventional verifier computes:

$$P \rightarrow V(P)$$

A recursive verifier computes:

$$P \rightarrow V_0(P) \rightarrow \text{Audit}(V_0) \rightarrow V_1(P) \rightarrow \text{Audit}(V_1) \rightarrow \dots$$

This allows the system to detect errors such as:

counterexample to stronger theorem $\rightarrow$ mistaken claim that original theorem is false

The verifier therefore becomes capable of identifying errors in its own reasoning path.

13 Existing Formal Proofs as Prior Evidence

Recursive verification should not automatically rebuild every theorem. Suppose repository A contains:

$$\text{theorem } T : P := \text{proof}_T$$

The verifier should record:

- Source repository
- Formal language
- Theorem statement
- Assumptions
- Dependencies
- Proof status

It should then determine whether T actually entails the proposition under review.

This avoids a second failure mode:

theorem already formally established $\rightarrow$ verifier does not locate it $\rightarrow$ verifier reports UNDER-SPECIFIED $\rightarrow$

A serious verification system must therefore perform **proof provenance analysis**.

14 Five-Language Verification

The proposed framework can represent canonical theorems across:

- Lean 4
- Coq
- Isabelle/HOL
- Agda
- F*

The objective is not syntactic identity. The objective is semantic equivalence. For theorem T :

$$T_{\text{Lean}}, \quad T_{\text{Coq}}, \quad T_{\text{Isabelle}}, \quad T_{\text{Agda}}, \quad T_{\text{F}^*}$$

must correspond to the same canonical proposition.

A theorem should not be marked universally verified if one implementation merely contains a placeholder or unproven postulate.

15 Refusal as a Formally Specified Operator

Refusal itself can be formalized.

Definition 12 (Formal Refusal). *Define:*

$$\text{Refusal}(x) = 1 \iff S(x)$$

where $S(x)$ is a policy predicate.

Then $\text{Refusal}(x) \leftrightarrow S(x)$ can itself become a formally specified proposition. But a second proposition must remain separate: $\text{Truth}(P)$.

Thus the system can verify $\text{Refusal}(x)$ without claiming $\neg P$.

A safety policy can say: *Do not provide operational instructions*. A theorem prover can simultaneously establish: *The mathematical identity is true*. Those statements are not contradictory.

16 Recursive Refusal Audit

A robust system should therefore permit the following sequence:

1. Sensitive terminology detected
2. Policy classification applied
3. Operational response constrained
4. Mathematical proposition extracted
5. Independent formal verification performed
6. Truth status determined

The policy layer controls what the system does. The verification layer determines what the mathematics establishes. Neither layer should impersonate the other.

17 Static Algebra Versus Dynamics

The same recursive method applies to dynamical claims. The equation:

$$y' = (W + \Delta W)x$$

defines a transformation. It does not automatically define:

$$x_{t+1} = F(x_t)$$

Therefore **threshold crossing** does not automatically imply **attractor**. An attractor requires a dynamical system. A verifier that discovers this distinction should report `DYNAMICS_UNDER_SPECIFIED` rather than `ALGEBRA_REFUTED`.

Again, the issue is proposition scope.

18 Proof Status Lattice

We propose the following statuses:



These statuses prevent the binary `true/false` classification from destroying important logical distinctions.

19 Experimental Research Program

A practical implementation should evaluate AI systems on a benchmark containing:

- Ordinary mathematical claims
- Sensitive-domain mathematical claims
- Ambiguous claims
- False universal claims
- True existential claims
- Under-specified dynamical claims
- Claims containing adversarial terminology
- Claims with valid counterexamples
- Claims with existing formal proofs

Measure:

- Refusal rate
- Proposition extraction accuracy
- Quantifier preservation
- Assumption preservation
- Counterexample validity
- Proof retrieval
- Self-correction rate
- False-refutation rate
- False-verification rate
- Policy/truth separation accuracy

The most important metric is **FALSE REFUTATION RATE**, defined as the proportion of cases where a system declares a proposition false even though a valid proof exists under the proposition’s stated assumptions.

20 Central Hypothesis

The research hypothesis is:

Recursive verification will reduce false refutation caused by proposition conflation, particularly when a request contains terminology capable of triggering a policy response.

This is an experimentally testable hypothesis. It should not be presented as an empirical fact until benchmark results establish it.

21 Conclusion

The fundamental problem is not refusal itself. Refusal can be an appropriate policy mechanism. The problem arises when refusal, classification, counterexample generation, and mathematical refutation are treated as equivalent operations. They are not.

A mature AI verification architecture should distinguish:

- **WHAT THE USER ASKED**
- **WHAT THE SYSTEM MAY PROVIDE**
- **WHAT THE MATHEMATICS ESTABLISHES**
- **WHAT THE SYSTEM HAS ACTUALLY PROVED**

The recursive principle is therefore:

Never allow a verifier's conclusion to escape verification merely because it came from the verifier.

A counterexample must be checked against the exact proposition. A refutation must preserve quantifiers. A theorem must preserve assumptions. A dynamical conclusion must contain dynamics. An existing formal proof must be discoverable and attributable. And a refusal must remain a policy decision rather than masquerading as a mathematical proof.

The resulting architecture is not a system that blindly answers every request. It is a system that knows the difference between:

- *I will not provide this*
- *this proposition is false*

That distinction is foundational to trustworthy AI reasoning.